Level 2

Hackathon – Fundamentals of Web Development

Project: Student Portfolio Website

Student Details

Student Name	K. Lubaina Zahara. S
Register Number	34625U50019
Department	B.Sc. (AI) – Artificial Intelligence
ID	CEO4D883872E23892A451AA21DEOFB6
College	M.M.E.S. Women's Arts and Science College
Date of Submission	27/09/2026

Git-Hub Link:

https://github.com/<your-username>/student-portfolio-website (replace with your repository link)

1. Problem Statement

Students usually share their details through paper resumes, long documents or scattered social-media profiles. These are difficult to update, do not look professional, and give recruiters and teachers no single place to see a student's education, skills, projects and achievements. A plain document also cannot filter projects, play animations or work well on a mobile phone.

A student portfolio website solves these problems. It presents personal information, educational qualifications, technical skills, projects and achievements in a professional and interactive format, can be opened from any device using one link, lets visitors filter projects, download the resume and contact the student through a validated form. The application is needed because it builds a strong online identity, is easy to update, and demonstrates the student's real web development skills.

2. Objectives

- To design a responsive and professional portfolio website using HTML, CSS and JavaScript.
- To provide About Me, Education, Skills and Projects sections with responsive navigation and smooth scrolling.
- To display project details dynamically using JavaScript and filter projects by technology or category.
- To provide a downloadable resume and a contact form with mandatory-field and email validation.
- To add interactive elements and basic animations, and to make the site work on desktop, tablet and mobile screens.

Expected outcome: a fully working front-end portfolio website that presents the student's profile in an interactive way without page reloads, and that demonstrates core web development concepts such as DOM manipulation, events, form validation, animations and responsive design.

3. Architecture

The application is a single-page website that runs entirely in the browser. It is divided into three layers: the Presentation Layer (HTML and CSS), the Application Logic Layer (JavaScript modules) and the Data Layer (skills and projects data held in JavaScript arrays).

Modules of the application

Module	Purpose	Main functions
Navigation Module	Responsive menu with a hamburger button on small screens, smooth scrolling to sections and highlighting of the active link.	menuBtn click, scroll listener
Skills Module	Reads the skills array and shows each skill with an animated progress bar.	renderSkills()
Projects Module	Creates project cards dynamically (description, technologies, links) and filters them by category.	renderFilters(), renderProjects()
Contact Module	Collects name, email and message; checks mandatory fields and email format; shows error or success message.	validateForm(), setError()
Effects Module	Typing animation, reveal-on-scroll animation, floating avatar and back-to-top button.	typeEffect(), IntersectionObserver

Technologies used and their roles

- **HTML5 –** Defines the structure of the page: header, hero banner, About, Education, Skills, Projects, contact form and footer.
- **CSS3 –** Styles the interface: colours, gradients, cards, grid layout, hover effects, animations and media queries for responsiveness.
- **JavaScript (ES6) –** Implements all logic: dynamic rendering of skills and projects, filtering, form validation, scroll effects and typing animation.
- **VS Code, Git –** Used for writing code, and Git / GitHub for version control and submission.
- **Web browser –** Any modern browser (Chrome, Edge, Firefox) is used to execute the application.

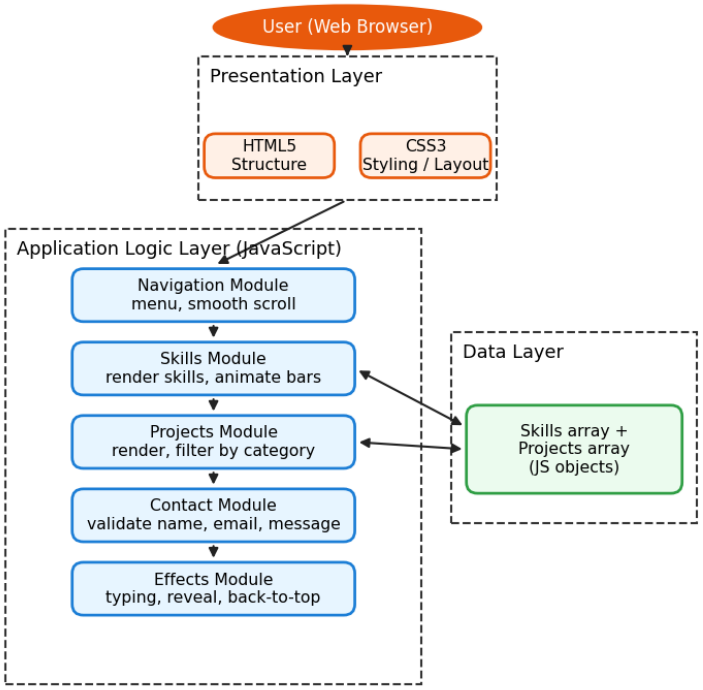


Figure 1: Application architecture and modules

User navigation and application flow

The visitor opens the Home page, uses the menu to move to the About, Education and Skills sections, browses the projects and filters them by category, can download the resume, and finally sends a message through the contact form, which is validated before a success message is shown.

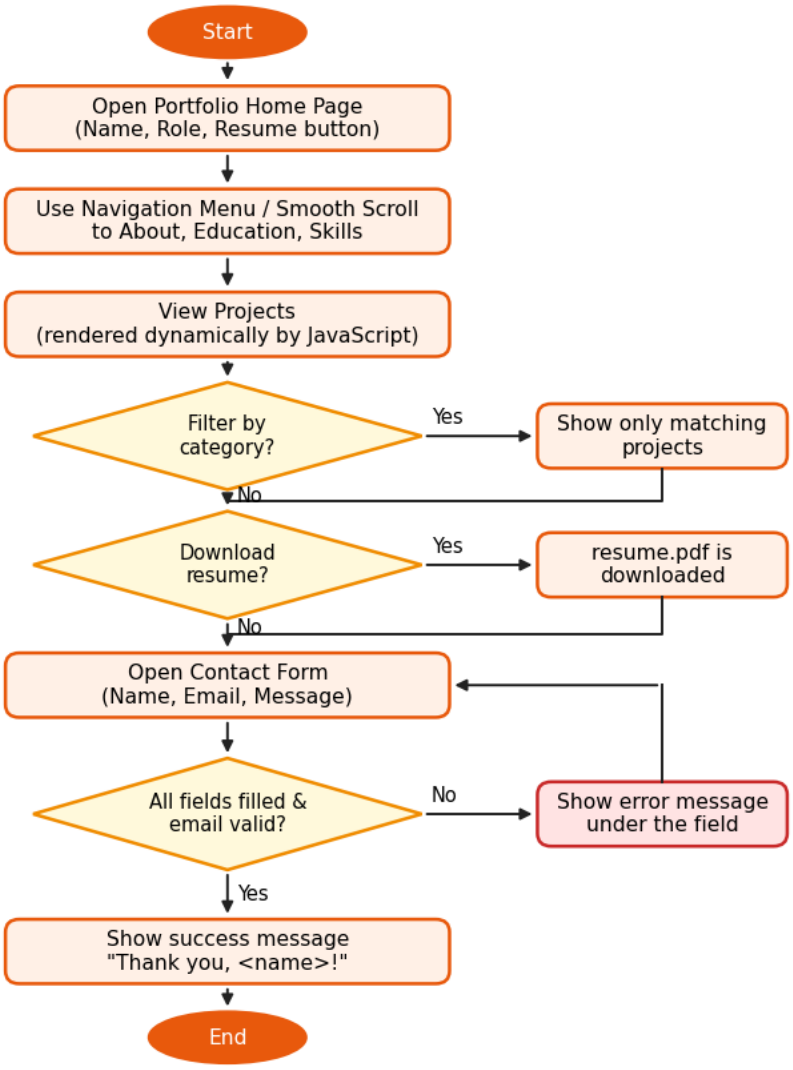


Figure 2: Application flow diagram (initial page to final output)

4. Practical Implementation

The application was developed step by step. First, the page layout was planned and the HTML skeleton was written with separate sections for home, about, education, skills, projects and contact. Next, CSS was added to give the site a purple gradient theme, card-based layout, animations and a responsive grid. Finally, JavaScript was written to store the skills and projects as arrays of objects, render them dynamically, filter the projects, validate the contact form and add scroll effects. Each feature was tested in the browser after it was built.

4.1 Features Implemented

Feature	Purpose
Responsive navigation	Fixed top menu with active-link highlight; becomes a hamburger menu on mobile screens.
About, Education, Skills, Projects	Separate sections that present personal details, qualifications, technical skills and projects.
Dynamic project display	Project cards are generated from a JavaScript array, so projects can be added or changed easily.
Project filtering	Category chips (All, Web, AI / ML, Python) show only the chosen type of project.
Downloadable resume	The Download Resume button downloads resume.pdf using the HTML download attribute.
Contact form validation	Name, email and message are mandatory; email format is checked with a regular expression.
Interactive elements and animations	Typing effect, animated skill bars, reveal-on-scroll, hover effects, floating avatar, back-to-top button.
Smooth scrolling	Menu links scroll smoothly to the selected section (CSS scroll-behavior).
Project links and descriptions	Every project card shows a description, technologies used, Source Code and Live Demo links.
Responsive design	CSS Grid, Flexbox and media queries adapt the layout to desktop, tablet and mobile screens.

4.2 Source Code

HTML

HTML gives the page its structure. The header holds the logo and navigation links, the hero section holds the name, animated role and resume button, and the *main* element holds the About, Education, Skills, Projects and Contact sections. The skills and projects containers are empty because JavaScript fills them. The contact form uses input and textarea elements with the *novalidate* attribute so that custom validation messages can be shown.

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Lubaina Zahara - Student Portfolio</title>
 <link rel="stylesheet" href="style.css">
</head>
<body>
 <header class="topbar">
 <a href="#home" class="logo">Lubaina<span>.</span></a>
 <button class="menu-btn" id="menuBtn" aria-label="Toggle menu">&#9776;</button>
 <nav id="nav">
 <a href="#home" class="nav-link active">Home</a>
 <a href="#about" class="nav-link">About</a>
 <a href="#education" class="nav-link">Education</a>
 <a href="#skills" class="nav-link">Skills</a>
 <a href="#projects" class="nav-link">Projects</a>
 <a href="#contact" class="nav-link">Contact</a>
 </nav>
 </header>

 <section id="home" class="hero">
 <div class="hero-text">
 <p class="hello">Hello, I am</p>
 <h1>K. Lubaina Zahara. S</h1>
 <h2>I am a <span id="typed"></span><span class="cursor">|</span></h2>
 <p class="lead">B.Sc. (AI) student who loves building clean, responsive websites.</p>
 <a href="#projects" class="btn">View Projects</a>
 <a href="resume.pdf" download class="btn outline">Download Resume</a>
 </div>
 <div class="avatar">KL</div>
 </section>

 <main>
```

```
<section id="about" class="reveal">
<h2 class="title">About Me</h2>
<div class="about-grid">
<p>I am a curious and hardworking student, currently studying B.Sc. Artificial
Intelligence. I enjoy web development, problem solving and turning ideas into working
applications. I want to build a career where AI and web technologies are used to solve
real problems.</p>
<ul class="facts">
<li><b>Name:</b> K. Lubaina Zahara. S</li>
<li><b>Register No:</b> 34625U50019</li>
<li><b>Course:</b> B.Sc. (AI)</li>
<li><b>Interests:</b> Web, AI, Python</li>
</ul>
</div>
</section>

<section id="education" class="reveal">
<h2 class="title">Education</h2>
<div class="edu-grid">
<div class="edu-card"><span class="year">Pursuing</span><h3>B.Sc. Artificial Intelligence</h3>
<p>M.M.E.S. Women's Arts and Science College</p></div>
<div class="edu-card"><span class="year">Completed</span><h3>Higher Secondary (HSC)</h3>
<p>Tamil Nadu State Board</p></div>
<div class="edu-card"><span class="year">Completed</span><h3>Secondary School (SSLC)</h3>
<p>Tamil Nadu State Board</p></div>
</div>
</section>

<section id="skills" class="reveal">
<h2 class="title">Skills</h2>
<div id="skillList" class="skill-grid"></div>
</section>

<section id="projects" class="reveal">
<h2 class="title">Projects</h2>
<div class="filters" id="filters"></div>
<div class="grid" id="projectGrid"></div>
</section>

<section id="contact" class="reveal">
<h2 class="title">Contact Me</h2>
<form id="contactForm" class="form" novalidate>
<label for="name">Name *</label>
<input id="name" type="text" placeholder="Your full name">
<small class="error" id="nameErr"></small>
<label for="email">Email *</label>
<input id="email" type="text" placeholder="you@example.com">
<small class="error" id="emailErr"></small>
<label for="message">Message *</label>
<textarea id="message" placeholder="Write your message here..."></textarea>
<small class="error" id="messageErr"></small>
<button type="submit" class="btn">Send Message</button>
<p class="success" id="formSuccess"></p>
</form>
</section>
</main>

<footer>&copy; 2026 K. Lubaina Zahara. S | M.M.E.S. Women's Arts and Science College</footer>
<button id="toTop" class="to-top" aria-label="Back to top">&#8593;</button>
<script src="script.js"></script>
</body>
</html>
```

CSS

CSS styles the application. It sets the colour theme, builds the fixed header and gradient hero banner, lays out cards with CSS Grid (auto-fill columns so it adapts to the screen width), and defines the skill bars, form styles and animations (@keyframes). Media queries at 900px and 700px make the layout suitable for tablets and mobiles. The class *.reveal* hides a section until JavaScript adds *.show* when it scrolls into view.

```
* { box-sizing: border-box; margin: 0; padding: 0; font-family: "Segoe UI", Arial, sans-serif; }
html { scroll-behavior: smooth; scroll-padding-top: 70px; }
body { background: #f6f4ff; color: #2b2850; line-height: 1.6; }
.topbar { position: fixed; top: 0; left: 0; right: 0; z-index: 10; display: flex; justify-content: space-between;
align-items: center; padding: 14px 50px; background: #fff; box-shadow: 0 2px 8px rgba(0,0,0,.1); }
.logo { font-size: 26px; font-weight: bold; color: #2b2850; text-decoration: none; }
.logo span { color: #6c4cf1; }
nav a { text-decoration: none; color: #2b2850; padding: 8px 15px; border-radius: 20px; font-size: 15px; }
nav a:hover, nav a.active { background: #6c4cf1; color: #fff; }
.menu-btn { display: none; background: none; border: none; font-size: 28px; cursor: pointer; color: #6c4cf1; }
.hero { display: flex; align-items: center; justify-content: space-around; gap: 30px; min-height: 100vh;
padding: 110px 60px 50px; background: linear-gradient(135deg, #5b3df5, #c026d3); color: #fff; }
.hello { font-size: 18px; opacity: .9; }
.hero h1 { font-size: 46px; line-height: 1.2; margin: 6px 0; }
.hero h2 { font-size: 26px; font-weight: 500; margin-bottom: 12px; }
#typed { color: #ffe066; font-weight: bold; }
.cursor { animation: blink 1s infinite; }
.lead { max-width: 480px; margin-bottom: 26px; }
```

```
.btn { display: inline-block; background: #ffe066; color: #2b2850; border: none; padding: 12px 26px; margin: 0 8px 8px 0;
border-radius: 25px; font-size: 15px; font-weight: 600; cursor: pointer; text-decoration: none; transition: .3s; }
.btn:hover { transform: translateY(-3px); box-shadow: 0 6px 14px rgba(0,0,0,.25); }
.btn.outline { background: transparent; color: #fff; border: 2px solid #fff; }
.avatar { width: 230px; height: 230px; border-radius: 50%; background: #fff; color: #6c4cf1; font-size: 82px;
font-weight: bold; display: flex; align-items: center; justify-content: center; animation: float 3s ease-in-out infinite;
      }
main { max-width: 1050px; margin: auto; padding: 0 20px; }
section.reveal { padding: 70px 0 20px; opacity: 0; transform: translateY(40px); transition: .8s; }
section.reveal.show { opacity: 1; transform: none; }
.title { font-size: 32px; text-align: center; margin-bottom: 30px; color: #2b2850; }
.title::after { content: ""; display: block; width: 60px; height: 4px; margin: 8px auto 0; background: #6c4cf1;
      border-radius: 3px; }
.about-grid { display: grid; grid-template-columns: 2fr 1fr; gap: 30px; align-items: center; }
.facts { list-style: none; background: #fff; padding: 20px; border-radius: 12px; box-shadow: 0 2px 8px rgba(0,0,0,.1); }
.facts li { padding: 5px 0; font-size: 15px; }
.edu-grid, .grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(300px, 1fr)); gap: 22px; }
.edu-card, .card { background: #fff; padding: 22px; border-radius: 12px; box-shadow: 0 2px 8px rgba(0,0,0,.1);
      transition: .3s; }
.edu-card:hover, .card:hover { transform: translateY(-6px); box-shadow: 0 8px 18px rgba(108,76,241,.25); }
.year { display: inline-block; background: #ede9fe; color: #6c4cf1; padding: 2px 12px; border-radius: 12px; font-size:
      13px; }
.edu-card h3, .card h3 { margin: 10px 0 4px; font-size: 18px; }
.skill-grid { display: grid; grid-template-columns: repeat(2, 1fr); gap: 20px 40px; }
.skill-top { display: flex; justify-content: space-between; font-weight: 600; font-size: 15px; margin-bottom: 5px; }
.bar { height: 10px; background: #e0dcf7; border-radius: 6px; overflow: hidden; }
.fill { height: 100%; width: 0; background: linear-gradient(90deg, #6c4cf1, #c026d3); border-radius: 6px; transition:
      width 1.5s ease; }
.filters { text-align: center; margin-bottom: 25px; }
.chip { border: 1px solid #6c4cf1; background: #fff; color: #6c4cf1; padding: 6px 18px; margin: 0 4px 6px;
border-radius: 18px; cursor: pointer; font-size: 14px; }
.chip.active, .chip:hover { background: #6c4cf1; color: #fff; }
.card { animation: fadeUp .6s; }
.card p { font-size: 14px; color: #555; margin-bottom: 10px; }
.cat { font-size: 12px; font-weight: bold; color: #c026d3; text-transform: uppercase; }
.tag { display: inline-block; background: #ede9fe; color: #5b3df5; padding: 2px 10px; margin: 0 4px 4px 0; border-radius:
      10px; font-size: 12px; }
.links a { font-size: 13px; font-weight: 600; color: #6c4cf1; margin-right: 14px; text-decoration: none; }
.form { display: flex; flex-direction: column; max-width: 560px; margin: auto; }
.form label { font-weight: 600; margin-top: 12px; font-size: 14px; }
.form input, .form textarea { padding: 12px; border: 1px solid #bbb; border-radius: 8px; font-size: 15px; margin-top:
      4px; }
.form textarea { height: 110px; resize: vertical; }
.form .invalid { border-color: #d92d20; background: #fff5f5; }
.error { color: #d92d20; min-height: 18px; font-size: 13px; }
.form .btn { background: #6c4cf1; color: #fff; align-self: flex-start; margin-top: 14px; }
.success { color: #2f9e44; font-weight: 600; margin-top: 8px; }
footer { margin-top: 70px; padding: 22px; text-align: center; background: #2b2850; color: #ddd; font-size: 14px; }
.to-top { position: fixed; right: 22px; bottom: 22px; width: 44px; height: 44px; border: none; border-radius: 50%;
background: #6c4cf1; color: #fff; font-size: 20px; cursor: pointer; display: none; }
.to-top.show { display: block; }
@keyframes blink { 50% { opacity: 0; } }
@keyframes float { 50% { transform: translateY(-14px); } }
@keyframes fadeUp { from { opacity: 0; transform: translateY(20px); } to { opacity: 1; transform: none; } }
@media (max-width: 900px) { .hero { flex-direction: column-reverse; text-align: center; padding-top: 100px; }
 .about-grid, .skill-grid { grid-template-columns: 1fr; } .hero h1 { font-size: 36px; } }
@media (max-width: 700px) { .topbar { padding: 12px 20px; } .menu-btn { display: block; }
 nav { display: none; position: absolute; top: 100%; left: 0; right: 0; background: #fff; flex-direction: column;
      padding: 10px 20px; }
 nav.open { display: flex; } .hero h1 { font-size: 30px; } .avatar { width: 160px; height: 160px; font-size: 56px; } }
```

JavaScript

JavaScript implements the application logic. The skills and projects arrays store the data. Functions render the skills and project cards into the page using template strings, the filter chips are handled by a single click listener (event delegation), validateForm() validates the contact form, an IntersectionObserver adds the scroll animations, and typeEffect() uses setTimeout to create the typing animation.

```
// ---------- Data ----------
const skills = [
 { name: "HTML5", level: 90 }, { name: "CSS3 & Responsive Design", level: 85 },
 { name: "JavaScript (ES6)", level: 75 }, { name: "Python", level: 80 },
 { name: "Git & GitHub", level: 70 }, { name: "Machine Learning Basics", level: 65 }
];
const projects = [
 { title: "TastyBites Food Ordering", cat: "Web", tech: ["HTML", "CSS", "JavaScript"],
 desc: "Online food ordering website with menu search, cart and order tracking.",
 code: "https://github.com/<your-username>/online-food-ordering", demo: "#" },
 { title: "Interactive Quiz App", cat: "Web", tech: ["HTML", "JavaScript"],
 desc: "Quiz application with timer, score calculation and result summary.",
 code: "https://github.com/<your-username>/quiz-app", demo: "#" },
 { title: "Student Marks Predictor", cat: "AI / ML", tech: ["Python", "Scikit-learn"],
 desc: "Predicts a student's final marks from study hours using linear regression.",
 code: "https://github.com/<your-username>/marks-predictor", demo: "#" },
 { title: "Sentiment Analyzer", cat: "AI / ML", tech: ["Python", "NLTK"],
 desc: "Classifies product reviews as positive, negative or neutral.",
 code: "https://github.com/<your-username>/sentiment-analyzer", demo: "#" },
 { title: "Expense Tracker", cat: "Python", tech: ["Python", "SQLite"],
```

```
 desc: "Command-line tool to record, search and summarise daily expenses.",
 code: "https://github.com/<your-username>/expense-tracker", demo: "#" },
 { title: "Student Portfolio Website", cat: "Web", tech: ["HTML", "CSS", "JavaScript"],
 desc: "This responsive portfolio that presents my profile, skills and projects.",
 code: "https://github.com/<your-username>/student-portfolio", demo: "#" }
];
let activeCat = "All";
const $ = (id) => document.getElementById(id);

// ---------- Navigation ----------
$("menuBtn").addEventListener("click", () => $("nav").classList.toggle("open"));
document.querySelectorAll(".nav-link").forEach(a =>
 a.addEventListener("click", () => $("nav").classList.remove("open")));

// ---------- Typing animation ----------
const roles = ["Web Developer", "AI Enthusiast", "Problem Solver", "Quick Learner"];
let r = 0, c = 0, deleting = false;
function typeEffect() {
 const word = roles[r];
 $("typed").textContent = word.slice(0, deleting ? --c : ++c);
 if (!deleting && c === word.length) { deleting = true; setTimeout(typeEffect, 1200); return; }
 if (deleting && c === 0) { deleting = false; r = (r + 1) % roles.length; }
 setTimeout(typeEffect, deleting ? 50 : 100);
}

// ---------- Skills Module ----------
function renderSkills() {
 $("skillList").innerHTML = skills.map(s => `
 <div class="skill"><div class="skill-top"><span>${s.name}</span><span>${s.level}%</span></div>
 <div class="bar"><div class="fill" data-level="${s.level}"></div></div></div>`).join("");
}

// ---------- Projects Module ----------
function renderFilters() {
 const cats = ["All", ...new Set(projects.map(p => p.cat))];
 $("filters").innerHTML = cats.map(c =>
 `<button class="chip ${c === activeCat ? "active" : ""}" data-cat="${c}">${c}</button>`).join("");
}
function renderProjects() {
 const list = projects.filter(p => activeCat === "All" || p.cat === activeCat);
 $("projectGrid").innerHTML = list.map(p => `
 <div class="card">
 <span class="cat">${p.cat}</span><h3>${p.title}</h3><p>${p.desc}</p>
 <div>${p.tech.map(t => `<span class="tag">${t}</span>`).join("")}</div>
 <div class="links"><a href="${p.code}" target="_blank">Source Code</a>
 <a href="${p.demo}" target="_blank">Live Demo</a></div>
 </div>`).join("");
}

// ---------- Contact Module (validation) ----------
const emailPattern = /^[^\s@]+@[^\s@]+\.[a-zA-Z]{2,}$/;
function setError(id, msg) {
 $(id + "Err").textContent = msg;
 $(id).classList.toggle("invalid", msg !== "");
 return msg === "";
}
function validateForm(e) {
 e.preventDefault();
 const name = $("name").value.trim(), email = $("email").value.trim(), msg = $("message").value.trim();
 $("formSuccess").textContent = "";
 let ok = setError("name", name ? "" : "Name is required.");
 ok = setError("email", !email ? "Email is required." :
 emailPattern.test(email) ? "" : "Enter a valid email address.") && ok;
 ok = setError("message", msg.length >= 10 ? "" : "Message must have at least 10 characters.") && ok;
 if (ok) {
 $("formSuccess").textContent = `Thank you, ${name}! Your message has been sent.`;
 $("contactForm").reset();
 }
}

// ---------- Scroll effects ----------
const observer = new IntersectionObserver(entries => entries.forEach(en => {
 if (!en.isIntersecting) return;
 en.target.classList.add("show");
 en.target.querySelectorAll(".fill").forEach(f => f.style.width = f.dataset.level + "%");
}), { threshold: 0.15 });

window.addEventListener("scroll", () => {
 $("toTop").classList.toggle("show", window.scrollY > 400);
 document.querySelectorAll("main section, #home").forEach(sec => {
 if (window.scrollY + 120 >= sec.offsetTop)
 document.querySelectorAll(".nav-link").forEach(a =>
 a.classList.toggle("active", a.getAttribute("href") === "#" + sec.id));
 });
});

// ---------- Events ----------
document.addEventListener("click", (e) => {
 if (e.target.dataset.cat) { activeCat = e.target.dataset.cat; renderFilters(); renderProjects(); }
});
$("toTop").addEventListener("click", () => window.scrollTo({ top: 0, behavior: "smooth" }));
$("contactForm").addEventListener("submit", validateForm);
```

```
renderSkills(); renderFilters(); renderProjects(); typeEffect();
document.querySelectorAll(".reveal").forEach(s => observer.observe(s));
```

4.3 Project Execution

How to run the application

- Download or clone the project folder from GitHub.
- Make sure the folder contains index.html, style.css, script.js and resume.pdf.
- Double-click index.html (or open it with Live Server in VS Code). No installation or internet connection is required.

Working process (user input to output)

- The visitor opens the website and sees the home page with the name, animated role and the View Projects and Download Resume buttons.
- The visitor clicks a menu link; the page scrolls smoothly to About, Education, Skills or Projects and the active link is highlighted.
- The skill bars and sections animate when they come into view.
- In the Projects section the visitor selects a category chip; only the matching project cards are displayed with description and links.
- Clicking Download Resume downloads the resume file.
- In the Contact section the visitor enters name, email and message and clicks Send Message.
- If any field is empty or the email is invalid, an error message is shown; otherwise a success message appears.

5. Output Screenshots

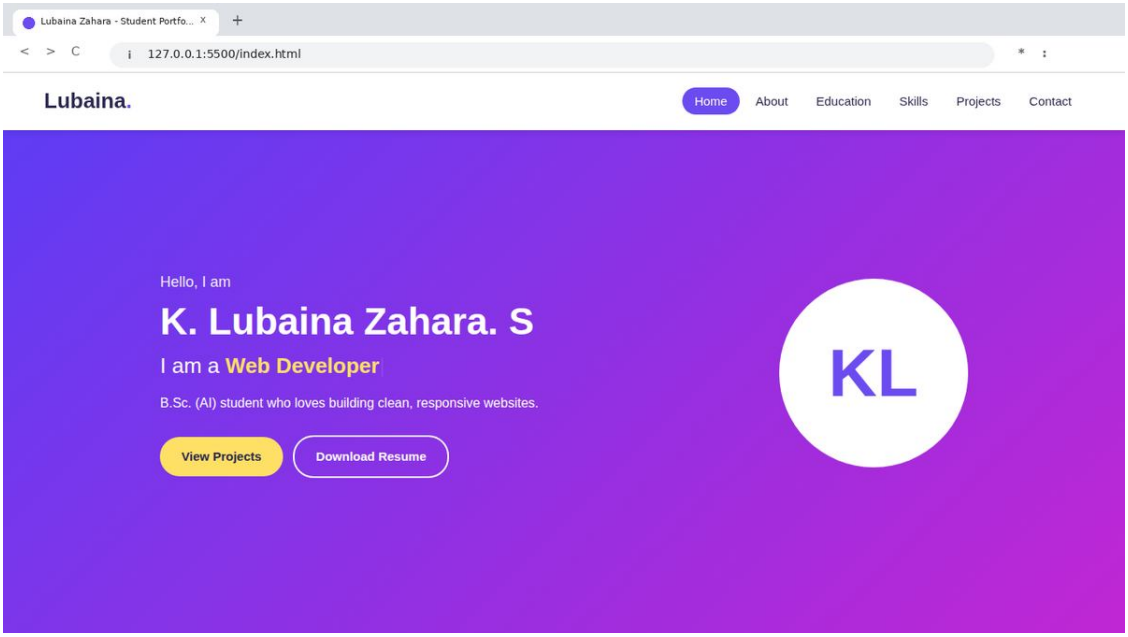


Figure 3: Home page with responsive navigation, typing animation and resume button

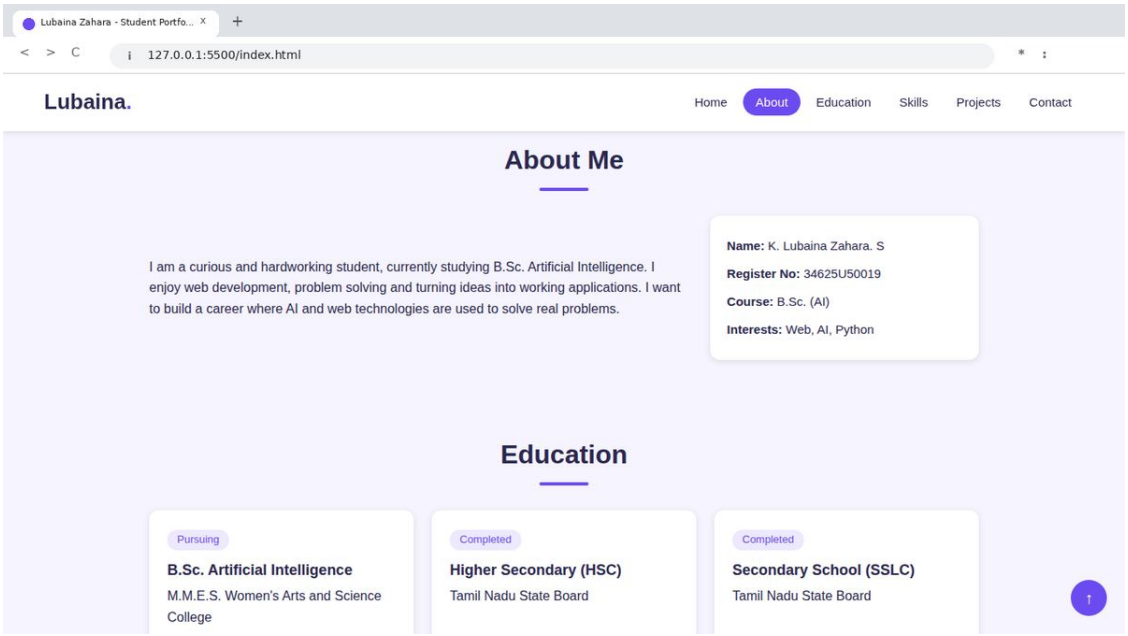


Figure 4: About Me and Education sections

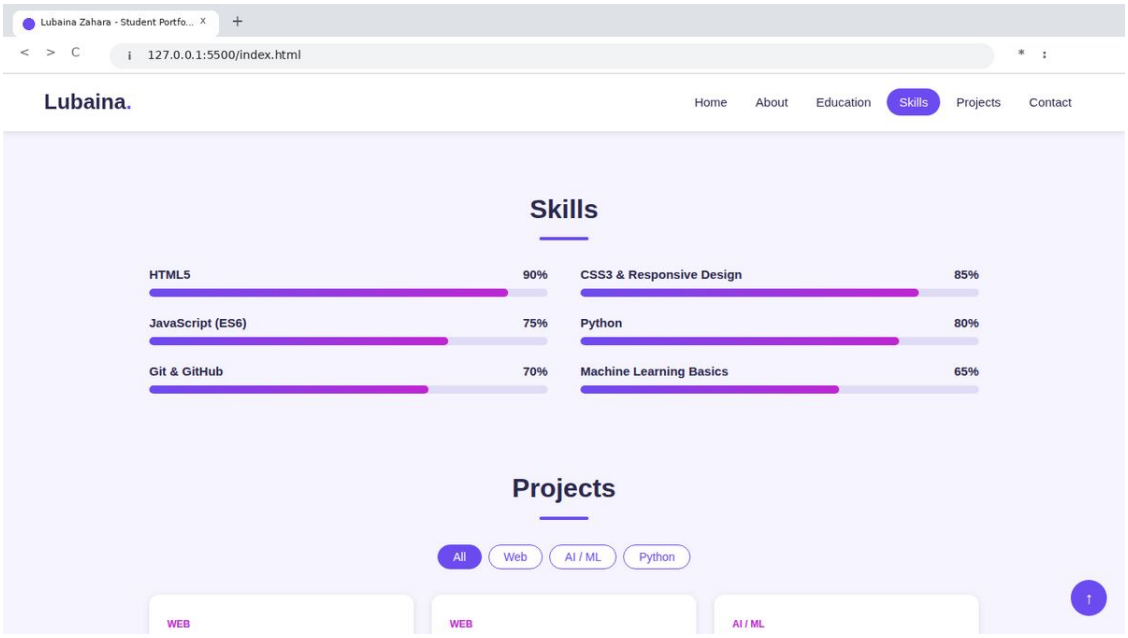


Figure 5: Skills section with animated progress bars

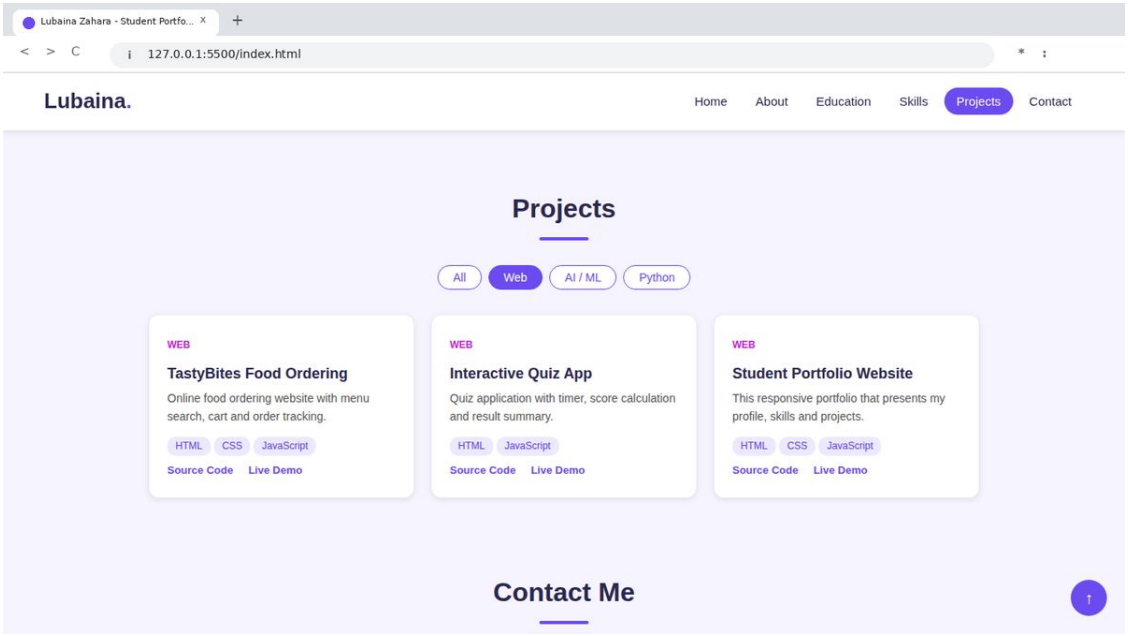


Figure 6: Projects section filtered by category (Web)

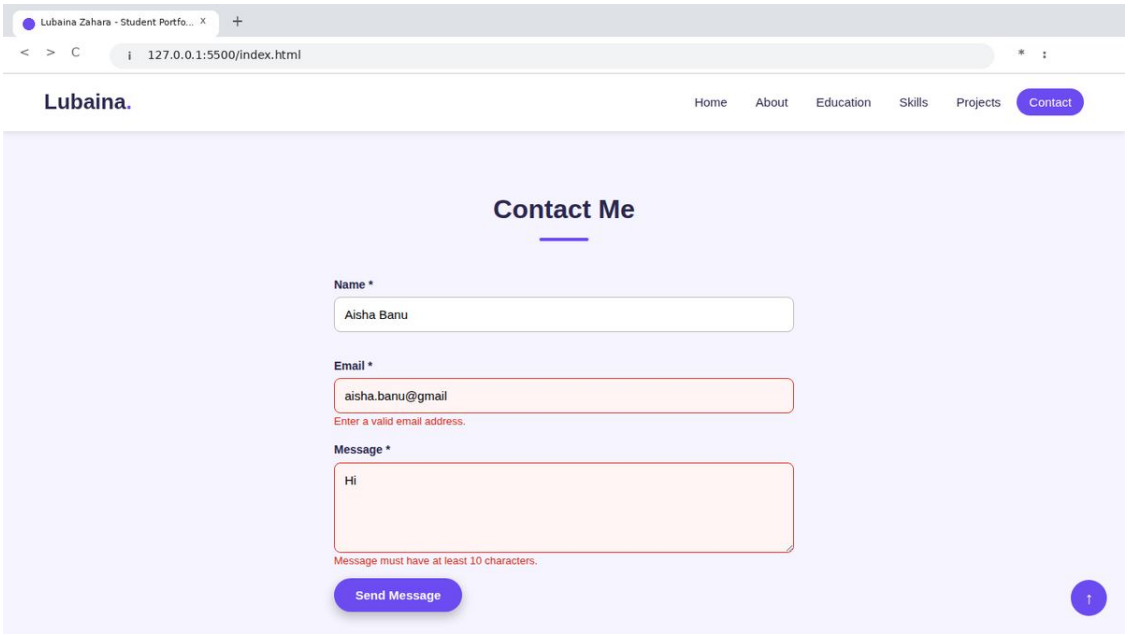


Figure 7: Contact form validation errors (invalid email and short message)

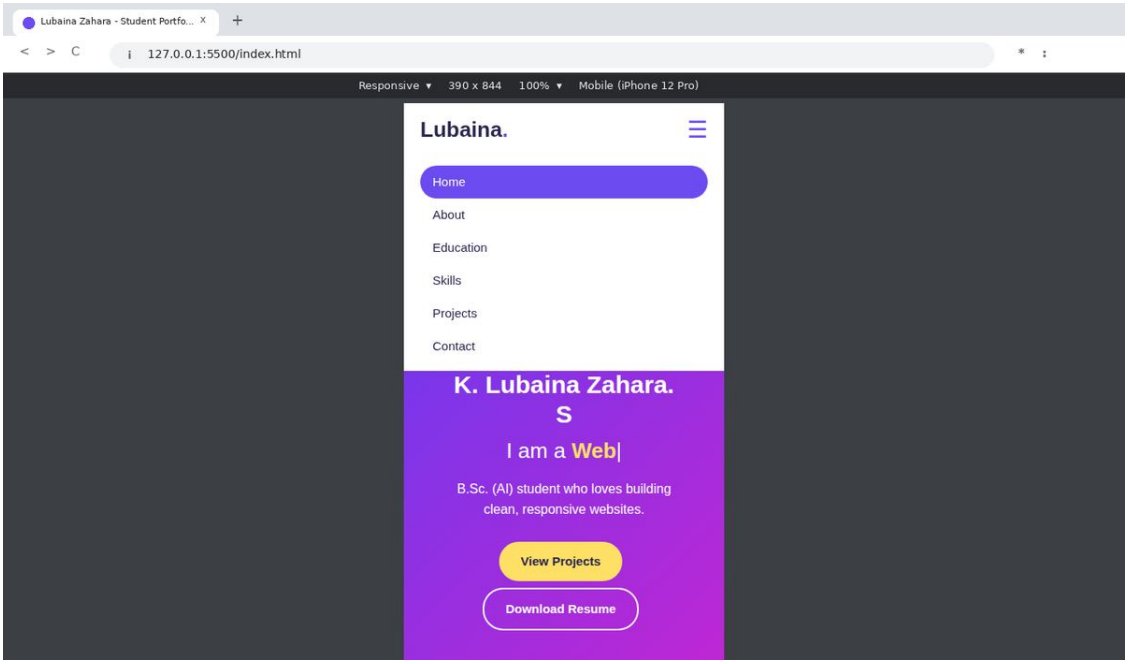


Figure 8: Responsive mobile view with hamburger menu

6. Technical Explanation

- **HTML5:** Semantic tags (header, nav, main, section, form, footer) organise the content and improve accessibility.
- **CSS3:** Flexbox is used for the header and hero; CSS Grid builds the responsive card layouts; gradients, shadows, transitions and @keyframes give a modern, animated look; media queries make the site responsive.
- **Data structures and array methods:** Skills and projects are arrays of objects. The methods map, filter and Set are used to build the cards, create the category list and apply the filter.
- **DOM manipulation:** Content is created with template strings and inserted using innerHTML; classes such as active, open and show are toggled with classList.

- **Event handling:** Event delegation handles the filter chips; click, scroll and submit events handle the menu, the back-to-top button and the contact form.
- **Validation:** Mandatory fields are checked and a regular expression validates the email address; error messages are shown below each field.
- **Animations and scroll effects:** IntersectionObserver reveals sections and fills the skill bars when they appear; setTimeout creates the typing effect; smooth scrolling uses CSS scroll-behavior.
- **Downloadable resume:** The anchor tag with the download attribute downloads resume.pdf without any server code.
- **Single page website:** All sections are on one page, so no page reloads happen when the user moves between sections.

7. Conclusion

The Student Portfolio Website was successfully developed using HTML, CSS and JavaScript. It presents the student's personal information, education, skills and projects in a professional and interactive format, filters projects by category, offers a downloadable resume, validates the contact form and works on desktop, tablet and mobile screens. Through this project I learned how to structure a web page, style it responsively, manipulate the DOM, handle events, validate forms and add animations. The project shows how the fundamentals of web development can be combined to build a practical, real-world application.

8. Future Enhancements

- Add a back-end (Node.js / Express with a database) to store contact messages and portfolio data permanently.
- Send the contact form message directly to the owner's e-mail using a service such as EmailJS.
- Add a dark / light theme switch and multiple language support.
- Add real profile photo, project screenshots, certificates and achievements gallery.
- Add a blog section and an admin page to add or edit projects without changing the code.
- Improve performance and SEO, and convert the site into a Progressive Web App.